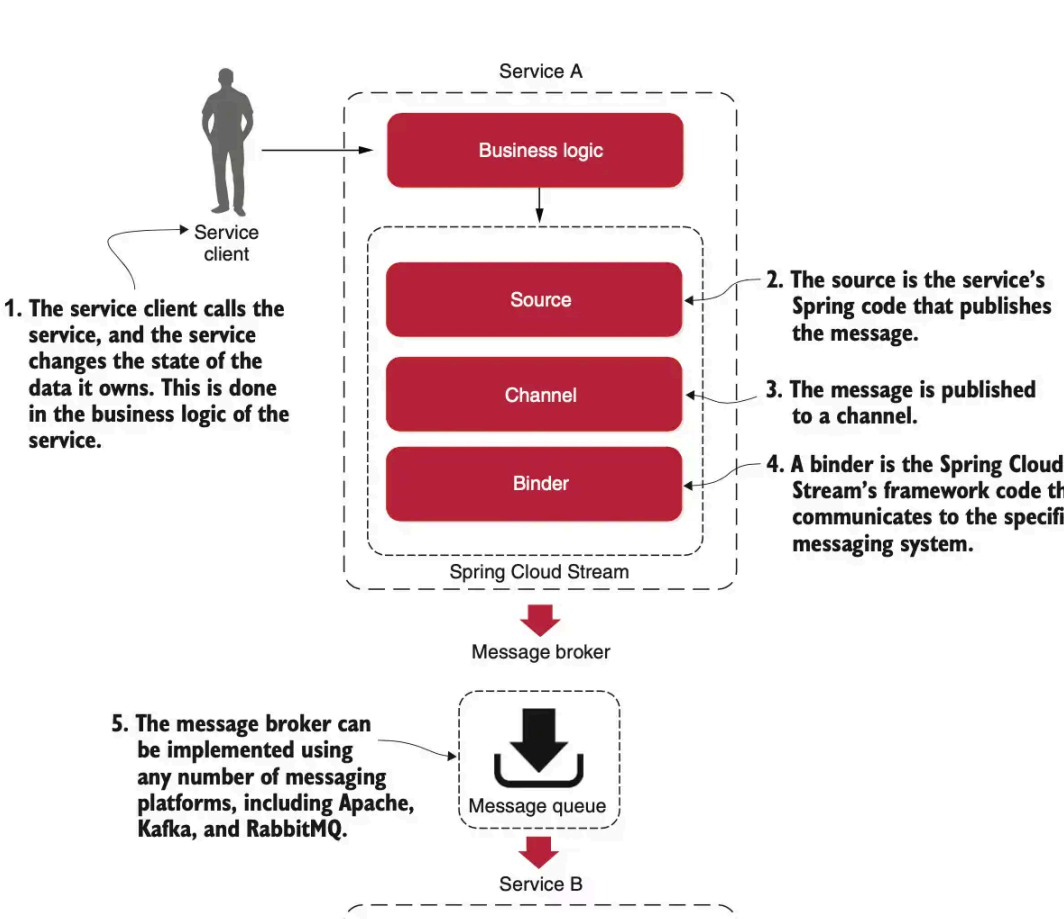
Asynchronous communications (Spring Cloud Stream)

Introduction

Spring Cloud Stream allows to abstract away the implementation details of the messaging platform (e.g., Apache Kafka, RabbitMQ). Thus, **implementation-specific details are kept out of the application code**. The publication and consumption of messages in applications is done through platform-neutral Spring interfaces.

Let's begin our discussion by looking at the Spring Cloud Stream architecture through the lens of two services communicating via messaging. One service is the **message publisher**, and one service is the **message consumer**.


Service A
Business logic
Service client
1. The service client calls the service, and the service changes the state of the data it owns. This is done in the business logic of the service.
Source
2. The source is the service's Spring code that publishes the message.
Channel
3. The message is published to a channel.
Binder
4. A binder is the Spring Cloud Stream's framework code that communicates to the specific messaging system.
Spring Cloud Stream
Message broker
5. The message broker can be implemented using any number of messaging platforms, including Apache, Kafka, and RabbitMQ.
Message queue
Service B
Spring Cloud Stream
6. The order of message processing (binder, channel, sink) changes as a service receives a message.
Binder
Channel
Sink
7. A sink is the service-specific code that listens to a channel and then processes the incoming message.
Business logic

- **Source**: Converts a POJO into a serialized message (default JSON) and publishes it to a channel.
- **Channel**: An abstraction over a queue; decouples the application from the actual queue name, allowing configuration changes without code changes.
- **Binder**: Connects to a specific messaging platform, letting the application send and receive messages without using platform-specific APIs.
- **Sink**: Listens on a channel, deserializes messages back into POJOs, and passes them to the service's business logic.

Project dependencies

Add `spring-cloud-stream`

Add at least one binder (e.g., `spring-cloud-starter-stream-rabbit` or `spring-cloud-starter-stream-kafka`).

The `<dependencyManagement>` section is typically used in Spring-based projects to manage Spring Cloud dependencies consistently. By using the `spring-cloud-dependencies` BOM, you can ensure that the correct dependencies are used and avoid version conflicts.

```
<properties>
    <spring-cloud.version>2025.1.0</spring-cloud.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-stream</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-stream-rabbit</artifactId>
    </dependency>
</dependencies>

<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring-cloud.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

Resources

- https://www.baeldung.com/spring-maven-bom